The Evidence: What We Proved in Part 2

The Evidence: What We Proved in Part 2 I

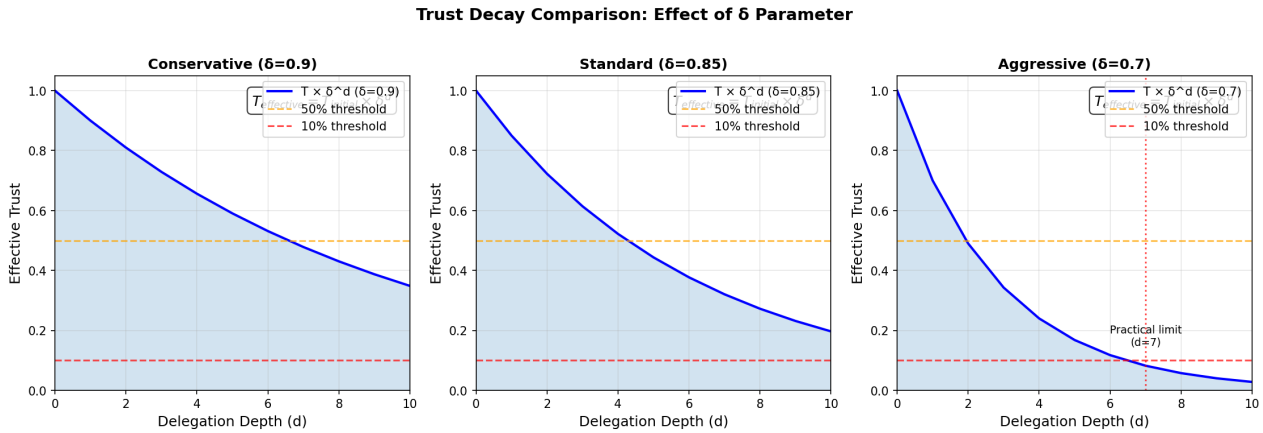

Figure 1: Effective trust vs. delegation depth at decay factors $\delta = \{0.9, 0.85, 0.7\}$, with 50% and 10% thresholds marking delegation bounds.

The Evidence: What We Proved in Part 2 I

Part 1 supplies the formal apparatus. Part 2 supplies the empirical evaluation: tested CIF modules, a **950-attack** corpus, and an architecture-aware simulation over four headline topologies (Claude Code, AutoGPT, CrewAI, LangGraph), with broader adapter coverage documented in Part 2.

Here is what the data says.

The Experimental Setup

The Experimental Setup I

We tested four common multi-agent architectures found in production:

The Experimental Setup I

- **Claude Code** (Hierarchical)
- **AutoGPT** (Autonomous Loop)
- **CrewAI** (Role-Based Team)
- **LangGraph** (State Machine)

The Experimental Setup I

The test corpus included direct prompt injection, poisoned RAG contexts, deep trust exploitation, and multi-turn social engineering.

Finding 1: Defense Layering vs. Individual Efficacy

Finding 1: Defense Layering vs. Individual Efficacy I

The Data: In the parametric evaluation the full CIF stack achieved a **96–100% parametric detection ceiling**, with direct injection detected at 99–100% across architectures. The separate real-pipeline evaluation had a lower multi-seed mean of approximately 44.8%. **The Implication**: The parametric model rewards layering, but the real pipeline does not spread the work evenly across layers, and the 100-attack ablation corpus says so plainly. The Invariants module alone detects 92.9% of that corpus against the full stack's 95.9%; the Firewall, Detection, Tripwire and Trust Calculus modules each detect between 2% and 6% on their own. Removing Invariants costs about 85 percentage points of true-positive rate, removing the Firewall or the Tripwire costs about one point each, and removing Consensus, Detection, Provenance, Sandbox or Trust Calculus costs nothing this corpus can

measure. That is a statement about *marginal* contribution, not about capability: a module whose detections are all also caught by Invariants scores zero here while detecting plenty on its own. Layering still buys coverage against attacks this corpus does not contain, but the older claim that removing any single layer opens a measurable gap is not what the ablation shows.

Finding 2: State Machine Determinism

Finding 2: State Machine Determinism I

The Data: **LangGraph** architectures achieved the highest detection rates (98%). **The Mechanism**: In a state machine, valid transitions are explicitly defined. Attempts to move from an `Analyze` state to an `Execute` state without passing through `Verify` were caught immediately. **The Implication**: Explicit state definitions provide a "security for free" effect, where the architecture itself enforces behavioral invariants that are difficult to bypass.

Finding 3: Hierarchical Vulnerability

Finding 3: Hierarchical Vulnerability I

The Data: **Claude Code** and **AutoGPT** styles were strong against external attacks but vulnerable to **Systemic (Ω_5)** compromise. **The Mechanism**: When the "Boss" agent was tricked, it ordered "Worker" agents to execute malicious actions, and they complied. **The Implication**: Hierarchical systems exhibit a Single Point of Failure at the root. Security in these topologies requires worker-level tripwires that can reject orders even from authenticated superiors.

Finding 4: Roles as Security Boundaries

Finding 4: Roles as Security Boundaries I

The Data: **CrewAI** performed exceptionally well against privilege escalation. **The Mechanism**: "Roles" acted as effective containers for capabilities. A "Researcher" agent lacked the tools to write code, rendering code-execution attacks against it inert. **The Implication**: Strict role definitions function as effective security boundaries, limiting the blast radius of a compromised agent.

Finding 5: The Stealth-Impact Tradeoff

Finding 5: The Stealth-Impact Tradeoff I

The Data: High-impact attacks were consistently easier to detect than low-impact nudges. **The Implication**: "Catastrophic" takeover attempts generate significant noise in the system state. The primary challenge for defenders is not the sudden takeover, but the slow, subtle drift of agent beliefs over long time horizons.

Finding 6: Emergent Misalignment Is the Hardest Scenario to Detect

Finding 6: Emergent Misalignment Is the Hardest Scenario to Detect I

The colony benchmark reveals a striking pattern: **emergent misalignment achieves the lowest detection rate (74.3%) of any evaluated scenario**, at a false positive rate of 25.5%. It is not the noisiest scenario: belief cascade detects every attack but at a higher false positive cost (37.4%). (These are Part 2's 30-seed benchmark means; an earlier single-seed figure of 56.1% is not the publication estimate.) Part 2's game-theoretic analysis does not explain why, and it is worth being exact about that. Its payoff matrix is a design model: thirty-five of its thirty-six cells have no measurement behind them, and the one that does is this scenario. On the published 74.3% the equilibrium moves to coordination with game value 0.61; it named emergent misalignment only while the matrix still carried the retracted single-seed 56.1%. What survives is the measurement itself: of the five

Finding 6: Emergent Misalignment Is the Hardest Scenario to Detect I

scenarios the colony benchmark actually runs, emergent misalignment is the hardest to detect, and by a clear margin.

Part 2's parametric evaluation and colony benchmark show that: - Full CIF achieves 99–100% detection against direct injection (Ω_1) and 96–100% against impersonation, the corpus category standing for trust exploitation (Ω_4) - Against emergent misalignment (distributed sub-threshold drift with no explicit adversaries), detection falls to 74.3% - A rational adversary, knowing CIF is deployed, is pushed away from direct injection. Where exactly it is pushed *to* is a design-model question rather than a measured one: on the published numbers Part 2's payoff matrix puts the equilibrium at coordination, not emergent misalignment

Finding 6: Emergent Misalignment Is the Hardest Scenario to Detect I

This is not a failure of CIF—it is a consequence of its success. When explicit attacks are reliably detected, adversaries are forced toward the subtlest and most distributed manipulation strategies. The 74.3% detection rate on emergent misalignment represents the current frontier of defensive capability, not a gap in the framework's design.

Operator implication: Deploy colony-scale entropy monitoring and schedule periodic manual behavioral audits (weekly for high-stakes deployments). The Ω_5 playbook (`sec:incident-response`) provides the response protocol when drift accumulates despite in-context detection.

Finding 7: The Implementation Gap Is a Feature, Not a Bug

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

The 10–11 percentage-point gap between the parametric ceiling (96–100%) and the real pipeline reflects **adapter implementation maturity**, not a failure of CIF's formal architecture. Its two ends are two different measurements and should not be read as one range around one deployment. The wide end is the distance from the parametric floor to the 30-seed pipeline mean of 86.3%. The narrow end is the distance from the top of the parametric range to the 95.9% the same pipeline now reaches on the 100-attack ablation corpus — a figure that stood at 12.2% until the Invariants module was rewritten, and that moved with no change to any other module. Plan against the wide end. Part~2 introduces a 5-level CMMI-style adapter maturity scale:

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

Level	Name	Marginal TPR	Description
1	Stub	$ $0%	Hardcoded scores; no domain logic
2	Heuristic	1–5%	Pattern matching; uncalibrated thresholds
3	Statistical	5–15%	Calibrated thresholds; regression features
4	Adaptive	15–30%	Online learning; per-architecture tuning
5	Verified	30%+	Formal guarantees; cross-architecture validated

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

The rubric column is each adapter's *marginal* contribution to true-positive rate, not a whole-pipeline detection rate: a Level-5 adapter is one that adds 30 or more percentage points when introduced, not one that reaches 30% detection on its own.

The current Claude Code adapter is at Level 3 (Statistical), explaining the 44.8% mean. The roadmap projects +35–41 percentage points of improvement by advancing adapters to Level 5 for the primary attack categories. The parametric ceiling (96–100%) represents what Level-5 adapters achieve—it is a design target, not an overclaim.

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

Operator implication: When deploying CIF, assess the maturity level of each adapter against your threat model. Level-3 adapters (current) provide meaningful protection against unsophisticated Ω_1–Ω_2 attacks; Level-4–5 adapters (planned) are required for Ω_4–Ω_5 protection. The gap is closeable—it is an engineering challenge, not a theoretical limitation.

A Note on the Numbers

A Note on the Numbers I

The detection rates in Part 2 are derived from a calibrated parametric simulation, modeled on the architecture's topology. They represent the *structural* security of the design.

A Note on the Numbers I

- **99.4% Overall Detection** means: "Across all architectures and attack categories, 99.4% of attack vectors are detected by the full CIF defense stack" (with architecture-specific rates ranging from 96–100%).
- It does **not** mean: "We have a magic Python script that catches 99.4% of all evil AI thoughts."

A Note on the Numbers I

We proved the *architecture* works. The implementation fidelity is the variable for the builder.

***A Note on Three Numbers**: Throughout this guide you will encounter three detection rates that may seem contradictory. They are not — they measure different things:*

- ***96–100%** (parametric simulation, $N = 3{,}800$): CIF's **design-level detection ceiling** — what the defense architecture achieves when adapters are fully mature (Level 5) and conditions match the calibrated model. This is the target, not the current reality.*
- ***44.8%** [95% CI: 43.2%, 46.4%] (multi-seed pipeline, 30 seeds): The **current empirical baseline** for the Claude Code architecture with Level-3 adapters. This is what you get today, out of the box, before adapter tuning.*
- ***95.9%** (100-attack ablation corpus, all categories including hardest): full-pipeline true-positive rate on a corpus built to include difficult attacks. This bullet read ~12.2% until the Invariants module was rewritten to score demand structure rather than topic nouns; no other module changed. Read it as an upper bound rather than a floor: the corpus is template-generated, a detector keyed on demand structure is being asked to recognise generated demands, and the 0% false-positive rate reported beside it comes from the fifty easy benign strings hard-coded in Part 2's ablation runner, not from the adversarially hard `BenignCorpus` behind the*

Tripwire Configuration Data I

The tripwire densities below are what Part 2's pipeline installs, not a target density. One `TripwireAdapter` is constructed per pipeline rather than per agent, and its constructor (`cogsec_multi agent_2_computational/src/composition/adapters.py`) adds exactly three canaries; `get_canary_count()` on the resulting adapter returns `{'identity': 1, 'boundary': 1, 'principal': 1, 'temporal': 0, 'general': 0}`. `add_temporal_canary` is defined in `src/core/tripwire.py` but never called, so no temporal canary is installed anywhere. An earlier revision of this table reported 3+ identity, 5+ boundary, 2+ principal and 1 temporal canary per agent;

Tripwire Configuration Data I

those densities were never run.

Tripwire Configuration Data I

Category	Count Used in Sim	Placement Strategy
Identity canaries	1 per pipeline	Core identity beliefs
Boundary canaries	1 per pipeline	Permission boundaries
Principal canaries	1 per pipeline	Trust relationships
Temporal canaries	0 (never installed)	Session continuity (unused)